FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Fri 30 September 2016 - Part Theory

NAME (capital letters pls.):

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

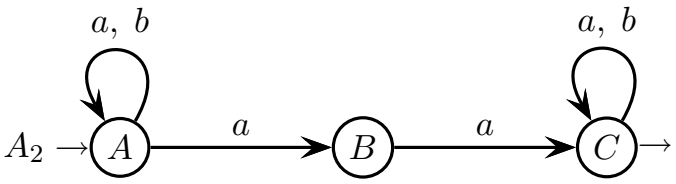
1. Consider a two-letter alphabet $\Sigma = \{\ a,\ b\ \}$ and answer the following questions:

   (a) Consider the following regular expression R_1 over the alphabet Σ:

   $$R_1 = (\ a\ b\ )^*\ (\ a\ a\ b\ )^*$$

   - By using the Berry-Sethi (BS) method, derive a deterministic finite-state automaton A_1 that accepts the language $L\,(R_1)$, and minimize it if necessary.
   - Is the language $L\,(R_1)$ local ? Give an adequate justification to your answer.

   (b) Consider the following finite-state automaton A_2 over the alphabet Σ.



   - By using the node elimination (BMC) method, derive a regular expression R_2 equivalent to the automaton A_2, and please make explicit every step of the derivation. Furthermore, briefly describe the characteristic feature of the strings that belong to language $L\,(A_2)$.
   - From the finite-state automaton A_2, derive an equivalent right-unilinear grammar G_2 that has the set of nonterminals $V_N = \{\ A,\ B,\ C\ \}$ (axiom A) associated with the states of A_2.
   - From the grammar G_2, derive an equivalent regular expression R_2' by solving an equation system in the variables L_A, L_B and L_C that denote the languages associated with the nonterminals of G_2. Use substitution and the Arden identity, and please make explicit every step of the derivation.

   (c) (optional) By using a systematic method, derive a finite-state automaton A_3 that accepts the intersection language $L\,(R_1)\ \cap\ L\,(A_2)$.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter terminal alphabet $\Sigma = \{ a, b \}$, the free grammar G (non-terminal alphabet $V = \{ S, B \}$ and axiom S) over Σ, and the two regular expressions R_1 and R_2 over Σ, as defined below:

$$G \begin{cases} S & \rightarrow & a\ S\ b \mid B \\ B & \rightarrow & B\ b \mid \varepsilon \end{cases}$$

$$R_1 = (\, a\ a \mid b\ b \,)^* \qquad\qquad R_2 = \left((\, a \mid b \,)^2 \right)^* = \left(\Sigma^2 \right)^*$$

Answer the following questions:

(a) Describe the language $L\,(G)$ as a string set, by giving its characteristic predicate, as follows:

$$L\,(G) = \{ \text{ string description} \mid \quad \text{characteristic predicate } \}$$

(b) Write a grammar G_1, *BNF* and unambiguous, that generates the intersection language L_1 below:

$$L_1 = L \cap L\,(R_1)$$

(c) Write a grammar G_2, *BNF* and unambiguous, that generates the intersection language L_2 below:

$$L_2 = L \cap L\,(R_2)$$

(d) (optional) Say if the complement language $\overline{L_1}$ is free or not, and justify your answer. In the case it is free, hint how to write a grammar $\overline{G_1}$ for it.

---

2. An mp3 player can process compilations that describe sound tracks and playlists. There are two musical genres: *classical* and *jazz*. Here are the language specifications:

   - A sound track consists of the following attributes, mandatory or optional:
     - the *genre*, denoted by the keywords **classical** or **jazz** (mandatory)
     - the *composer*, a string (mandatory)
     - the *performer*, a string (optional)
     - the *title*, a string (mandatory)
     - the *duration*, two integers for minutes and seconds (both mandatory)
     - the *size* in KBytes, an integer number (mandatory)
   - A playlist consists of a non-empty list of sound tracks and/or nested playlists, according to the following rules:
     - sound tracks and playlists can occur in any order
     - there are three types of playlist: *miscellaneous*, *classical* and *jazz*
     - a playlist of type miscellaneous can include sound tracks of any genre and playlists of any type
     - a playlist of type classical or jazz can include only sound tracks of the same genre and playlists of the same type

   A compilation consists of a non-empty list of sound tracks and playlists, in any order. Here is a sample compilation:

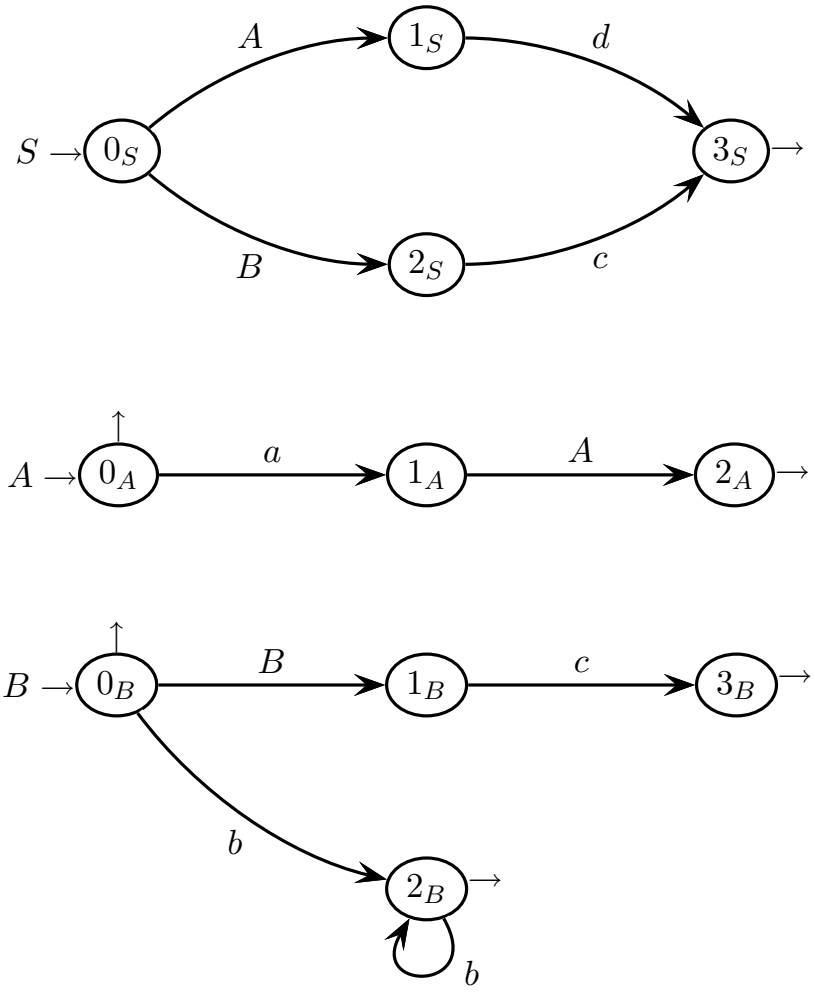
```
classical track
  composer Brahms  performer Levy  title Ballade  min 3 sec 41  size 4782
end track
miscellaneous playlist
  jazz track
    composer Davis  title SoWhat  min 2 sec 44  size 6423
  end track
  classical playlist
    classical track
      composer JSBach  performer Kremer  title Aria
      min 3 sec 34   size 7239
    end track
    classical playlist
      classical track
        composer WAMozart  performer Bollini  title prelude
        min 5 sec 21  size 8451
      end track
      classical track
        composer Vivaldi  title concert in F  min 12 sec 45  size 42683
      end track
    end playlist
  end playlist
end playlist
```

   Write a grammar G, *EBNF* and unambiguous, that models the sketched language.

---

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G, represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the three-letter nonterminal alphabet $V = \{ S, A, B \}$ (axiom S).



Answer the following questions:

(a) Draw the complete pilot of grammar G, say if grammar G is of type $ELR(1)$ and shortly justify your answer. If the grammar is not $ELR(1)$ then highlight all the conflicts in the pilot.

(b) Write all the guide sets on the arcs of the machine net (shift and call arcs, and final arrows), say if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer. If you wish, you can use the figure above to add the call arcs and annotate the guide sets.

(c) (optional) Say if the language $L(G)$ is of type $ELL(1)$, and in the case it is provide a grammar G' that has the $ELL(1)$ property.

4 Language Translation and Semantic Analysis 20%

1. Let string w_k denote the binary encoding of the natural number k (in the following natural numbers are represented in decimal notation, unless otherwise specified), and let string w_k^R denote the mirror image of w_k. For instance:

$$w_{12} = 1\,1\,0\,0 \qquad \text{and} \qquad w_{12}^R = 0\,0\,1\,1$$

Consider the following translation function τ, defined on the binary strings of type w_k^R, such that it holds $k > 0$ (i.e., the encoded natural number is strictly positive) and such that the binary encoding does not have any leading zeros:

$$\tau\left(w_k^R\right) = w_{k+1}$$

Here are a few translation samples:

- $\tau\,(0\,0\,1\,1) = 1\,1\,0\,1$ as $w_{12} = 1\,1\,0\,0$ (the encoding of 12 is $1\,1\,0\,0$) and $w_{13} = 1\,1\,0\,1$
- $\tau\,(1\,1) = 1\,0\,0$ as $w_3 = 1\,1$ (the encoding of 3 is $1\,1$) and $w_4 = 1\,0\,0$
- $\tau\,(0\,0\,1) = 1\,0\,1$ as $w_4 = 1\,0\,0$ (the encoding of 4 is $1\,0\,0$) and $w_5 = 1\,0\,1$
- $\tau\,(1\,1\,0\,1) = 1\,1\,0\,0$ as $w_{11} = 1\,0\,1\,1$ (the encoding of 11 is $1\,0\,1\,1$) and $w_{12} = 1\,1\,0\,0$

On the other hand:

- $\tau\,(0) =$ *undefined* because the argument is zero
- $\tau\,(1\,1\,0) =$ *undefined* because of the leading zero in the source string

A simple *BNF* grammar that generates the source language is as below (axiom L):

$$\begin{cases} L & \rightarrow & 0\ L \\ L & \rightarrow & 1\ L \\ L & \rightarrow & 1 \end{cases}$$

Answer the following questions:

(a) Write a *BNF* translation grammar G_τ, or scheme, that defines the above described translation τ. If necessary you can change the source grammar.

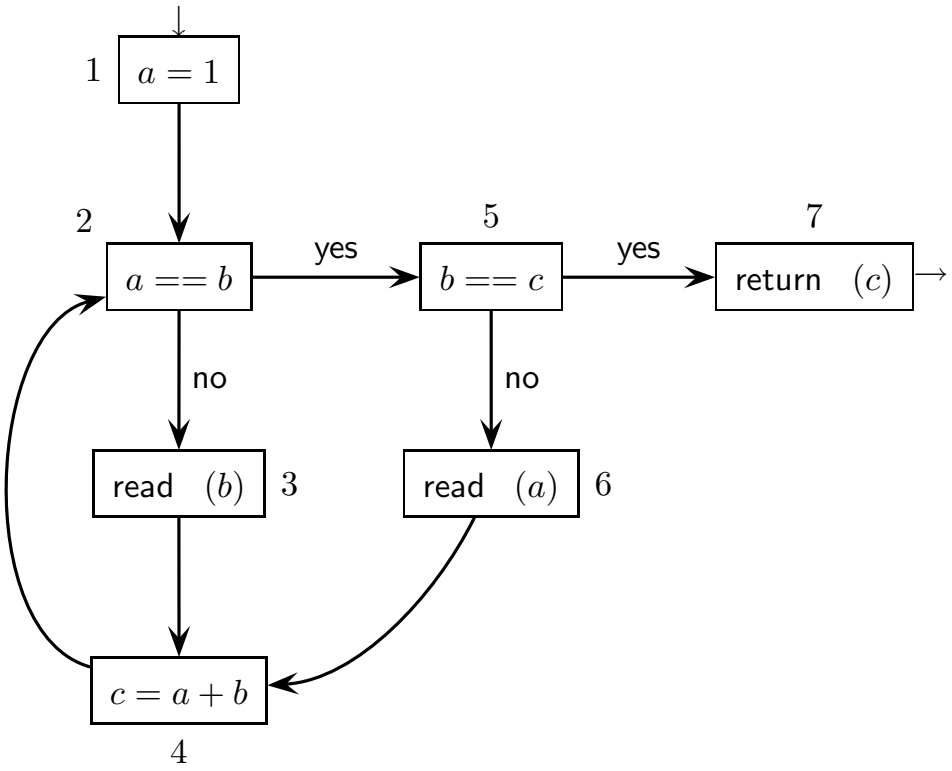
(b) With reference to grammar G_τ, draw the syntax trees for the following translation samples: $\tau\,(1) = 1\,0$, $\tau\,(1\,1) = 1\,0\,0$ and $\tau\,(1\,1\,0\,1) = 1\,1\,0\,0$

(c) Argue that grammar G_τ is such that the translation samples $\tau\,(0)$ and $\tau\,(1\,1\,0)$ are both undefined.

(d) (optional) Is translation τ deterministic ? Briefly justify your answer.

---

2. Consider the following control flow graph (*CFG*) of a simple program:



Variables b and c are input parameters, and variable c is also output parameter, while variable a is local.

Answer the following questions:

(a) Find the *live variables* of the given *CFG*, by directly applying the definition of live variable. Write the live variables on the *CFG* prepared on the next pages.

(b) Find the *reaching definitions* of the given *CFG*, by means of the systematic method of the flow equations for reaching definitions: first write the equations, then iteratively solve them (use the tables prepared on the next pages). Write the reaching definitions on the *CFG* prepared on the next pages.

---

please here write the **LIVE VARIABLES** (at the node inputs) obtained directly

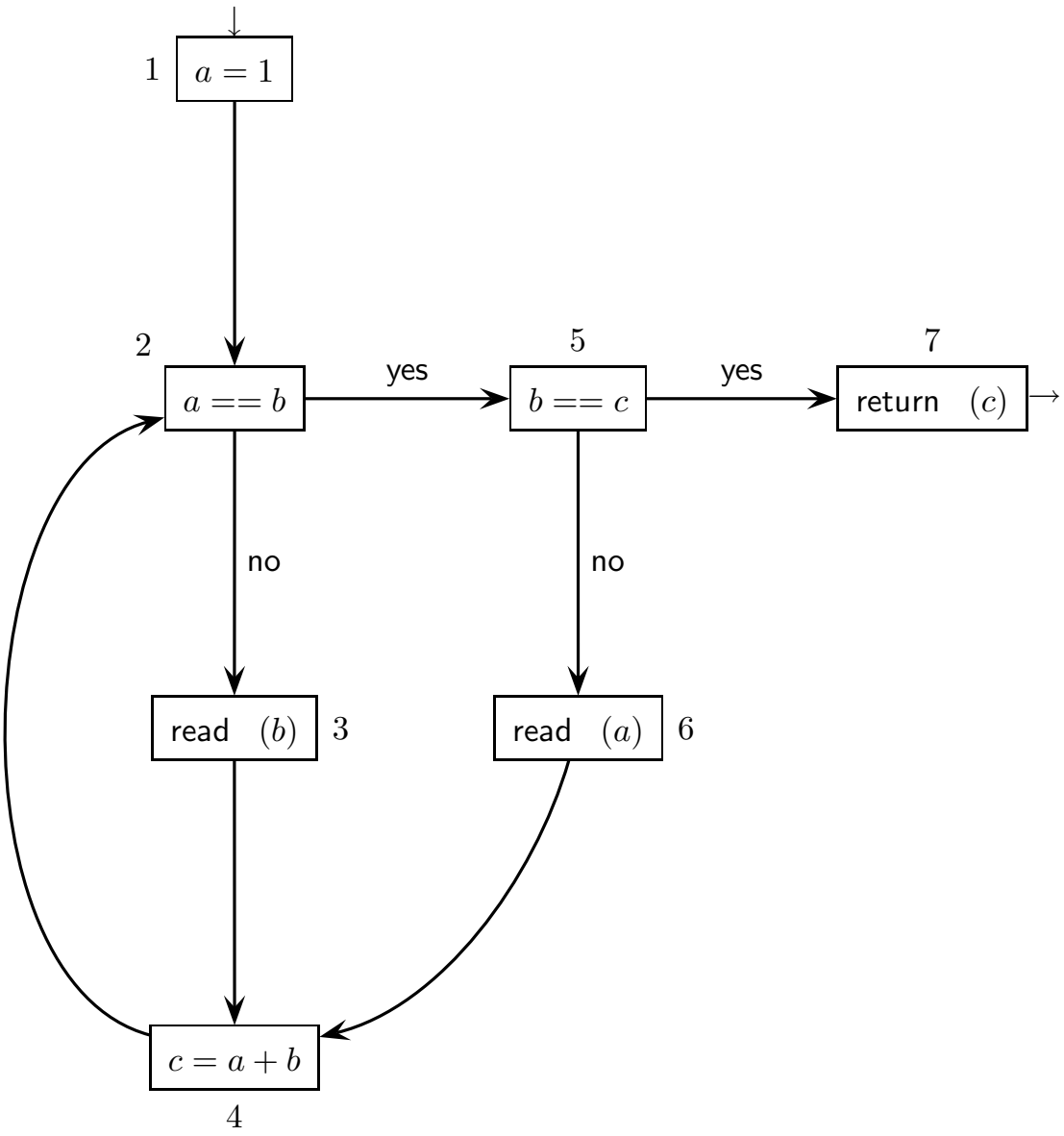


table of definitions and suppressions at the nodes
for **REACHING DEFINITIONS**

node	*defined*
1	
2	
3	
4	
5	
6	
7	

node	*suppressed*
1	
2	
3	
4	
5	
6	
7	

system of data-flow equations for **REACHING DEFINITIONS**

node	*in equations*	*out equations*
1		
2		
3		
4		
5		
6		
7		

iterative solution table of the system of data-flow equations (**REACHING DEF.S**)

(the number of tables and columns is not significant)

	initialization		1		2		3		4		5		6	
#	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*
1														
2														
3														
4														
5														
6														
7														

	7		8		9		10		11		12		13	
#	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*	*in*	*out*
1														
2														
3														
4														
5														
6														
7														

please here write the **REACHING DEFINITIONS** (at the node outputs)
obtained systematically